

2025-後学期

# 令和7年度 プログラミング演習 グループプログラミング レポート

## 「ネットワーク対戦型2Dシューティング ゲーム」

|         |         |
|---------|---------|
| プログラム   | メディア情報学 |
| クラス     | J1      |
| グループ番号  | 19      |
| 2411516 | 池崎 文哉   |
| 2411540 | 大谷 華景   |
| 2411548 | 奥野 棕太   |
| 2411730 | 南 悠莞    |

# 1 プログラムの概要

どのような目的のプログラムを実現したか、どのような機能を実装したか、をまず最初に説明してください。必要に応じてスクリーンキャプチャした画像も張り付けてください。

「ドローエディタ」ならば、どのような機能を持ったドローエディタを作ったか説明してください。作業の分担など、作業をどのように進めたかも説明して下さい。

---

## 1.1 プログラムの目的

私たちのグループでは、対戦型の 2D シューティングゲームを作成した。本作品における目的は機体を操作し球を発射して相手に当てて相手の HP をゼロにすることである。

## 1.2 プログラムの主な仕様

本作品では、ゲームアプリを立ち上げた後、部屋を建てるか部屋に参加するかを選び対戦を開始することができる。対戦開始後、プレイヤーはスペースキー、Z キー、X キーによって異なる 3 種類の球を発射できる。マップ上にはランダムな位置にアイテムが生成され、取得すると回復効果や弾丸のパワーアップ効果を得られる。ゲーム終了後はリザルト画面に遷移し、そこで両者のプレイヤーがリトライを選択すると再戦することができる。

## 1.3 プログラムの解析

「1.2 プログラムの主な仕様」よりプログラムの実現のために必要な処理は大まかに以下のように分割できる。

- プレイヤークラスの実装
- 弾丸クラスの実装
- ギミッククラスの実装
- プレイヤーと弾丸やギミックの接触判定を確認する管理システム
- サーバーとクライアント間での通信の確立および実行
- 画面遷移
- リトライ機能
- ユーザーのキー入力を取得する機構

## 1.4 作業の分担

作業の分担として、池崎は全体のゲームの動きを統括する Model を作成した。また、画面揺れや通信の軽量化など既存のコードの改良も行った。大谷はプレイヤー、弾、ギミックなどのゲームを構成する要素の実装を行った。南は画面遷移やリトライ機能の実装、通信の確立を担当した。奥野はコントローラーの機能の実装や背景画像の素材の用意を主に行った。

---

## 2 設計方針

プログラムの基本構想を述べる。どのようなアルゴリズム，データ構造を用いて，目的の処理を実現するのか説明する。なぜ，そのようなアルゴリズム，データ構造を用いたかという考察も含める。ここでは，プログラムの細部には触れない。

今回は，Java のプログラミングなので，どのようなクラスを用意して，どのように利用するかを説明する。クラス図を使用してください。発表会のプレゼンで作成したクラス図を流用して構いません。

例を 図 1 に示す。

---

「1.3 プログラムの解析」での解析を元にプログラムの基本構想について設計方針を述べる。

### 2.1 プレイヤークラスの実装

### 2.2 弾丸クラスの実装

### 2.3 ギミッククラスの実装

### 2.4 プレイヤーの当たり判定の管理

ゲームにおけるプレイヤー、弾丸、ギミックの位置情報の管理は原則としてサーバー側が行う。サーバー側では両プレイヤーの位置座標が記憶されており入力をコントローラーから受け取ることによってその位置座標を逐次更新する。また弾丸及びギミックの位置は Iterator によって管理され、それらすべてに対してプレイヤーとの距離を計算することでヒット確認を行う。

### 2.5 通信の確立

### 2.6 通信の実行

本ゲームはネットワーク型の対戦ゲームであるため、ゲームの状態を両プレイヤー間で共有し共通の状態にしておかなければならない。そのために、まずサーバー側でゲームの状態を完全に管理する。そしてサーバー側で計算された各物体の情報を StringBuilder を用いて文字列としてサーバー側からクライアント側へと伝える。また、通信の軽量化を行うために受信専用スレッドによってデータを受信し、得られた情報が上書きされて失われないようにキューに入れてそこから取り出して情報を読み取るようにする。

### 2.7 画面遷移

### 2.8 キー入力の取得

### 2.9 クラス

クラス図は以下の図 1 のようになった。

---

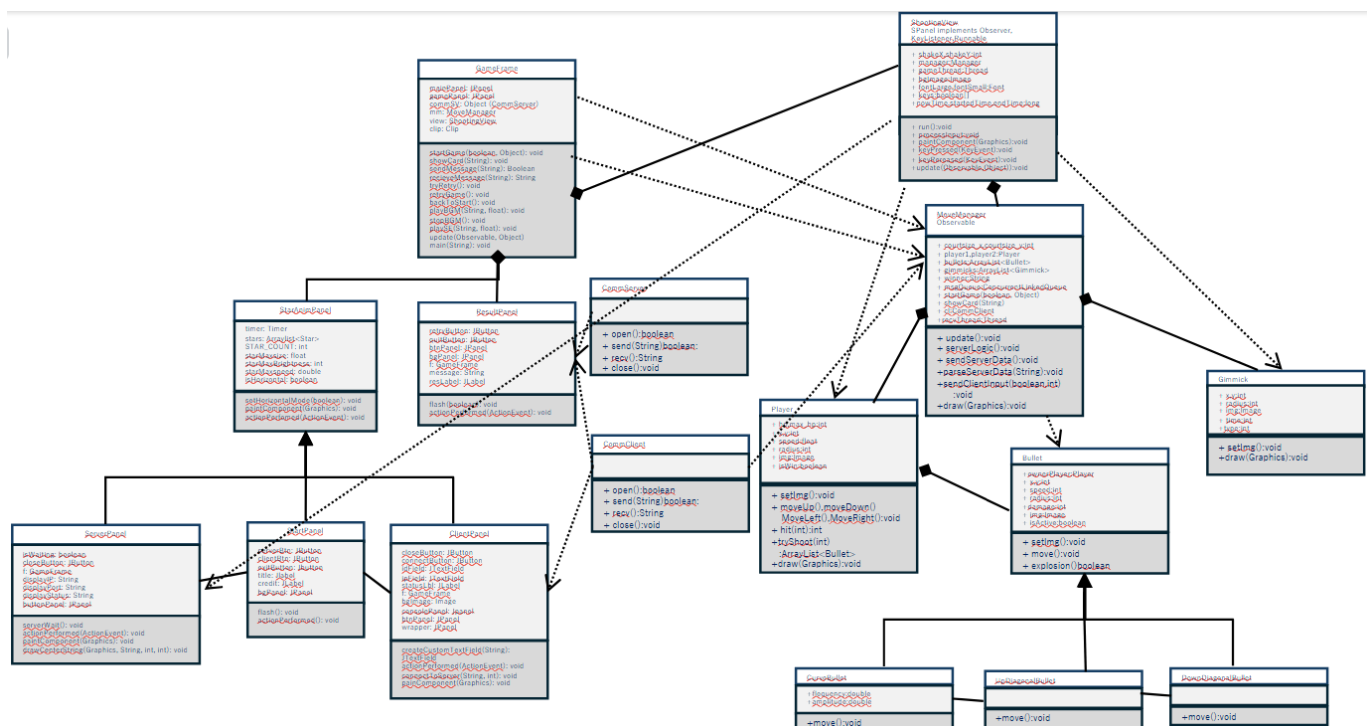


図 1: クラス図.

### 3 プログラムの説明

メンバー全員が各自の担当した部分を説明. 工夫点を述べる.

実際に作成した主要なクラスの主なメソッド, フィールドの説明を行う. 必要に応じてプログラムリストを抜粋して説明する.

#### 3.1 池崎

#### 3.2 大谷

【Bullet クラスと, その子クラス CurveBullet, UpDiagonalBullet, DownDiagonalBullet】

Bullet クラスは, 弾の基本情報を管理するクラスである. 弾の座標, 速度, 描画, 移動処理, および爆発アニメーションの状態管理を行う. また, CurveBullet(曲線弾), UpDiagonalBullet, DownDiagonalBullet(斜め弾) は, Bullet クラスを継承し, 移動方法を定義した move() メソッドや弾の画像を変更したものである. 全フィールドから抜粋した主なフィールドは次の通り.

- x, y: 弾の座標
- speed: 弾の速度
- radius: 弾の半径
- img: 弾の画像
- state\_explosion: 弾の爆発状態. 通常時は 0 で, 5 まで段階的に変化する.

- **AnimationFrames:** 爆発中において、その画像が表示されて何フレーム目か。

抜粋した主なメソッドは次の通り。

- `Bullet(Player owner, int x, int y, float speed, int radius, int damage, int Shootdir, Color color)`  
サーバー用のコンストラクタ
- `Bullet(int x, int y, int radius, Color color, int Shootdir, int state_explosion, int AnimationFrames):`  
クライアント用のコンストラクタ。受け取った `state_explosion` の値によって、`radius` と `img` に代入する画像を決定する。
- `move()`: 弾の移動を定義している。Bullet では x 方向への直線的な移動、CurveBullet ではそれに加えて y 方向への `Math.sin` を用いた移動、Up(Down)DiagonalBullet では y 方向への直線的な移動を処理している。
- `explosion()`: 弾の爆発による画像、半径変更を処理。 `state_explosion` と `AnimationFrames` の値によって画像を決定。それぞれの段階の画像は `AnimationFrames` が 0~4 の間表示され、それ以降は次の段階にうつり、`AnimationFrames` は 0 にもどる。爆発が進行したかを `MoveManager` クラスで認識するため、次のように `if` 文、`else if` 文の中身が実行されたときのみ `true` を返し、そうでないときは `false` を返す。

```

1      public boolean explosion() {
2          if (this.state_explosion == 1) {
3              this.img = new ImageIcon(getClass().getResource("/images/BulletExplosion1.png")).getImage();
4              this.radius = 20;
5              return true; //進行したため、true を返す。
6          } else if (this.state_explosion == 2 && this.AnimationFrames == 5) {
7              //(中略)
8          }
9          return false; //進行していないため、false を返す。
10     }

```

工夫点—それぞれの継承クラスの `move()` メソッドは、次の通り Bullet クラスのものをオーバーライドしたものであり、同様の作業で容易に新たな特殊弾を実装することができる (例は CurveBullet)。

```

1      public void move() {
2          super.move(); //Bullet の move() 呼び出し

3          //追加の動き
4          // sin の引数更新
5          time += 0.5;
6          // y 座標更新 (爆発状態でなければ)
7          if (super.getStateExplosion() == 0) {
8              int newY = (int) (y + Math.sin(time * frequency) * amplitude);
9              super.setY(newY); // y 座標セット
10         }
11     }

```

## 【Player クラス】

プレイヤーキャラクターの操作、情報 (体力、パワーアップ状態など) の管理、および描画を管理するクラス。弾の生成処理も行う。抜粋した主なフィールドは次の通り。

- `hp, max_hp`: 現在の体力、最大体力

- x, y: プレイヤーの座標
- speed: プレイヤーの速度
- radius: プレイヤーの当たり半径
- img: プレイヤーの現在の画像
- biggerbullet: ギミック効果時の弾の半径増加量. 通常時は 0 に設定されている.
- statePowerup: ギミック効果を受け始めて何フレームか. 通常時は 0.

抜粋した主なメソッドは次の通り.

- `Player(int hp,int max_hp,int x,int y,float speed,int bounds_x_min,int bounds_x_max, int bounds_y,int radius, int Shootdir, int biggerbullet, int statePowerup):`  
コンストラクタ
  - `setImg()`: プレイヤーの画像を決定する. `statePowerup` が 0 ならば通常の画像を設定, そうでなければ効果時用の画像を設定する. 次のような  $(statePowerup / 10) \% 2$  の値による分岐によって, 効果中 2 つの画像をフレームによって切り替える.
- ```

1      public void setImg() {
2          if (Shootdir == 1 && statePowerup == 0) { // サーバー側, 通常時
3              this.img = new ImageIcon(getClass().getResource("/images/player1T.png")).getImage();
4          } else if (Shootdir == 1 && statePowerup > 0) { // サーバー側, 弾半径増加時
5              if ((statePowerup / 10) % 2 == 0) { // フレームによる分岐で効果中 2 つの画像を切り替える
6                  this.img = new ImageIcon(getClass().getResource("/images/player1PowerupT1.png")).getImage();
7              } else {
8                  this.img = new ImageIcon(getClass().getResource("/images/player1PowerupT2.png")).getImage();
9              }
10         } else if (Shootdir != 1 && statePowerup == 0) {
11             //(中略)
12         }
13     }

```

### 3.3 南

### 3.4 奥野

---

## 4 実行例

実行画面を示す. 結果のみでなく, 分かりやすく説明する.

必要に応じてグラフや図などを用いる.

例を 図?? に示す.

今回の場合は, グラフの代わりにスクリーンショットをつけて, 説明すること. 特に, アピールしたい点については丁寧に実行例を説明すること.

## 5 考察

完成したプログラムに対する考察，コメントを述べてください．当初予定していた通りの物ができたか考察してください．また，考察を踏まえて，今後の改良点についても述べてください．

---

### 5.1 プログラムについての考察

### 5.2 ゲームシステムについての考察

今回作成したゲーム作品が多くの人に評価してもらえ、最優秀賞をとることができたのは対戦型 2Dシューティングゲームというコンテンツ自体の面白さの面が大きかったと考える。2D 対戦型のシューティングゲームはゲームの構造としてシンプルながら奥深さがあるため、プレイしたいと思ってもらいやすかった。しかしながら今回作成したゲームでは、2D 対戦型シューティングゲームの土台を作ることになり苦戦したため、このゲーム独自の奥深さを十分に出すことができなかった。当初の予定では、フィールドを何種類か作成しダメージギミックなどそのステージ独自のギミックを作成する予定であったのでその部分は改善点である。また、今回はデバッグの時間をあまりとれていないため本来意図していない動作をした時などでバグが発生する可能性もある。これらの点をこれから改善していきたい。

---

## 6 各自の反省と感想

以下のような内容に関して，メンバ全員がそれぞれ書いてください．メンバ間で内容が重複しても構いません．必ず，それぞれ誰の感想か分かるようにしてください．

- グループでの作業を通しての反省や感想，それから考察できること．
  - 今後の各自の担当部分の課題．やり残したこと．
  - Java やオブジェクト指向，MVC モデルなど学習内容に関する感想．
  - 「プログラミング演習」の授業に関する感想や要望．
- 

### 6.1 池崎

私たちのグループでは、コード作成の初めにクラス図を作成して分担を行うことができていなかったため並行して作業することがあまりできておらず作業の効率化が行えていなかった。一人での作成であればファイル構造が複雑化してもある程度問題なく開発できるが、グループでのプログラム作成や保守性を考えると、もっとファイル構造の簡略化を行い可視性を上げる必要があると思う。そのためにも、オブジェクト指向についてもっと考える必要があると考える。ゲーム独自の要素としてのギミックの種類を増やすことが今後の課題である。また、リトライ機能において片方が RETRY を押しもう片方は QUIT を押すなど予期しない動作を行った時の動きの正常化など技術力が足りず直しきれなかったバグもあったのでこれらも改善したい。Java 言語などのオブジェクト指向言語では MVC モデルなどでの構造の簡

略化が非常に大事だと感じた。プログラミング演習の授業を通してグループでのプログラミングを行い開発の難しさやファイル構成を事前に決めることに大切さなどを学ぶことができた。

---



## 付録1：操作法マニュアル (ユーザーズマニュアル)

そのプログラムを初めて使う人向けの説明書を書いてください。ドローエディタなら操作法の説明、ゲームならキー操作の説明などを書いてください。スクリーンキャプチャした画像に説明を書き込んだりしてもいいでしょう。2 ページ程度の簡潔な説明書で構いません。

なお、付録は始まる直前で必ず改ページして下さい。

—————。java GameFrame をターミナル上で実行することでアプリが立ち上がる。ゲームを立ち上げたら、プレイヤーのうちどちらかが CREAT ボタンを押して部屋を立てる。もう片方のプレイヤーは JOIN ボタンを押し、部屋を立てた側のプレイヤーの画面に表示されている IP アドレスと ROOM ID(ポート番号) をクライアント側の画面に入力することで通信が確立し対戦が始まる。ゲーム中、プレイヤーは矢印キーで上下左右に動くことができる。また、スペースキーを押して直進弾、Z キーで波弾、X キーで斜め弾を発射することができる。ゲーム中にはフィールド上にアイテムが生成される。図??のようなアイテムを拾うと、発射する弾の大きさが図??のように大きくなる。また、図??のようなアイテムを拾うと体力が回復する。どちらかの体力が 0 になればゲーム終了となりリザルト画面に遷移する。リザルト画面ではゲームを終了するか再戦するかを選ぶことができ、両方のプレイヤーが RETRY を押すことで再戦ができる。意図しない挙動を引き起こす可能性があるため、両者の選ぶボタンは異なるものにする。

—————

## 付録2：プログラムリスト

プログラムリストは本文中には説明に必要な部分だけ抜粋して掲載して、プログラムリスト全体はレポートの最後に「付録」として付ける。プログラムコードには、主なメソッド、フィールドの説明など最低限のコメントは付ける。

cat -n hash.c などとして、行番号付きのプログラムリストを生成して、  
`\begin{verbatim}....\end{verbatim}`を用いて記述すると読みやすい。ページ数が多くなる場合は、`\small`や`\footnotesize`を用いるとよい。

```
1 struct item* insert(struct item s){
2     int i; struct entry *p, *chain;
3     i=h(s.key);
4     chain=H[i];
5     if(chain){
6         while(chain && comparekeytype(chain->term.key, s.key)) chain=chain->next;
7         if(chain) return &(amp;chain->term); /* 同じ鍵が見つかった 場合*/
8     }
```